

ScanGUI v6

Guide utilisateur et presentation technique complete

Projet C++ / GTK / API locale / PostgreSQL / Offline / Web

Juillet 2026

Objectif du document

Ce guide est conçu pour un développeur externe au projet. Il présente l'utilisation du logiciel, l'architecture, les flux d'exécution, les fichiers importants et les fonctions centrales avec des extraits de code copiés depuis le projet.

Table des matières

1	Resume executif	2
2	Etat des evolutions demandees	2
3	Guide d'utilisation	3
3.1	Mode local direct	3
3.2	Mode serveur local	3
3.3	Client web local	3
3.4	Mode offline	3
3.5	OCR et recherche texte	3
4	Architecture de code	3
5	Extraits de code commentes	3
5.1	1. Contrat commun de donnees : ScanDataSource	3
5.2	2. Etat et widgets principaux de la fenetre GTK	5
5.3	3. Initialisation des callbacks de telechargement	6
5.4	4. Barre superieure : mode local, serveur, offline	6
5.5	5. Choix dynamique de la source active	7
5.6	6. Bibliotheque : recherche, tri, favoris, cartes	7
5.7	7. Lecture : page simple, double page, webtoon, zoom	8
5.8	8. Marque-pages, historique et recherche OCR dans GTK	9
5.9	9. Synchronisation offline depuis le serveur	10
5.10	10. File de jobs asynchrones	11
5.11	11. Execution thread-safe des jobs	12
5.12	12. Copie offline complete d'une bibliotheque	13
5.13	13. Source locale : bibliotheque, profils et recherche texte	14
5.14	14. Source API : scans, images, progression, OCR	16
5.15	15. Routage central de l'API	18
5.16	16. API de bibliotheque paginee et filtree	19
5.17	17. Recherche OCR et resume de chapitre	20
5.18	18. Securite admin locale et token optionnel	20
5.19	19. Schema PostgreSQL enrichi	20
5.20	20. Indexeur OCR local / sidecar / Tesseract	21
5.21	21. Client web local minimal	22

5.22 22. Tests unitaires ajoutés	24
6 Verification realisee	25
7 Limites restantes assumées	25

1 Resume executif

ScanGUI est maintenant organise comme une application multi-sources : une interface GTK, un serveur HTTP local C++, une base PostgreSQL, une option web locale et une synchronisation offline. Le coeur de l'evolution consiste a faire passer l'application par l'abstraction ScanDataSource, ce qui permet de lire les memes scans depuis le disque local, depuis le serveur API ou depuis une copie offline.

Architecture cible

```

GUI GTK / Client web / futur CLI
|
v
ScanDataSource ou API REST
|
+-- FileSystemScanDataSource -> scan/ + cache/profiles
+-- ApiScanDataSource         -> ScanUIServer -> PostgreSQL + scan/
+-- OfflineLibrarySync        -> copie locale lisible hors connexion

```

2 Etat des evolutions demandees

Evolution	Etat	Commentaire
Mode client / serveur local	OK	La GUI propose Local direct, Serveur local et Offline. 'ApiScanDataSource' consomme l API, met les pages en cache et sauvegarde la progression.
Bibliotheque visuelle	OK	Cartes de scans, recherche, tri, favoris, profils, avancement et bouton reprendre.
Experience de lecture	OK	Plein ecran, double page, webtoon, zoom molette, déplacement souris, barre de progression, slider, miniatures, raccourcis, marque-pages, historique.
Telechargement en tache de fond	OK	'DownloadQueue' gere worker thread, progression, retry, pause, resume et annulation cooperative.
Serveur local securise	OK	Mutex autour du controleur, token admin optionnel, admin localhost uniquement, CORS configurable, endpoint stats/version, controle de chemin image.
JSON	Limite	'SimpleJson' centralise le parsing sans ajouter de dependance. Une migration vers 'nlohmann/json' reste possible mais n est pas imposee.
Tests unitaires	OK	'make tests' et 'ctest' passent. La suite couvre session, JSON, source locale, offline, queue, favoris, bookmarks et recherche sidecar.
Profils	OK	Profils, progression separee, favoris, marque-pages et historique existent cote API/BDD et sont visibles dans GTK.
OCR et recherche intelligente	OK	Indexation de fichiers texte sidecar et Tesseract optionnel. Recherche et resume simple. Pas de segmentation IA avantee des bulles.
Version web locale	OK	Client HTML/CSS/JS servi par le serveur C++ avec bibliotheque, lecteur, favoris, bookmarks et recherche OCR.

Evolution	Etat	Commentaire
Option offline	OK	Synchronisation des scans du serveur vers 'scan/' via 'OfflineLibrarySync', puis lecture en mode Offline.

3 Guide d'utilisation

3.1 Mode local direct

Place les images dans la structure scan/<nom_du_scan>/<numero_chapitre>/<numero_page>.jpg. Lance ensuite l'application GTK sur une machine disposant de gtkmm-3.0 et choisis Local direct. La progression, les favoris et les marque-pages locaux sont stockés dans cache/profiles/<profil>.

```
make
./bin/ScanGUI
```

3.2 Mode serveur local

Le serveur lit le dossier scan/, indexe la bibliotheque en PostgreSQL et expose une API REST. Dans GTK, choisis Serveur local, renseigne http ://127.0.0.1 :8787, puis clique sur Actualiser.

```
docker compose -f docker-compose.postgres.yml up -d
export SCANGUI_HOST=127.0.0.1
export SCANGUI_PORT=8787
export SCANGUI_SCAN_ROOT=scan
export SCANGUI_ADMIN_TOKEN=dev-token
make server
./bin/ScanGUIServer
```

3.3 Client web local

Le serveur expose aussi un client web minimal. Ouvre http ://127.0.0.1 :8787/ dans un navigateur. Le web consomme les memes routes que ApiScanDataSource.

3.4 Mode offline

En mode Serveur local, clique sur Sync offline. Le job materialise toutes les pages dans scan/. Ensuite, bascule la GUI sur Offline pour lire sans serveur.

3.5 OCR et recherche texte

Le mode OCR le plus fiable consiste a placer un fichier texte a cote de chaque image, par exemple scan/Serie/1/1.txt pour 1.jpg. Si SCANGUI_TESSERACT_CMD est defini, l'indexeur peut aussi appeler un moteur externe.

```
curl -X POST -H "x-scangui-admin-token: dev-token" http://127.0.0.1:8787/api/admin/ocr
curl "http://127.0.0.1:8787/api/search?q=mot"
```

4 Architecture de code

Le projet est separe en couches : domaine, application, interface GTK, serveur, web et tests. La couche application fait le lien entre les ecrans et les sources de donnees. Le serveur garde la responsabilite de PostgreSQL, de l'indexation et de la securite des routes HTTP.

5 Extraits de code commentes

5.1 1. Contrat commun de donnees : ScanDataSource

Ce contrat est le point d'entree de toute la bascule local / serveur / offline. La fenetre GTK n'a plus besoin de savoir si les pages viennent du disque ou de l'API.

include/application/ScanDataSource.hpp - lignes 15-106

```

#include <filesystem>
#include <string>
#include <vector>

struct ScanSummary {
    std::string id;
    std::string title;
    int chapterCount{0};
    int pageCount{0};
    ScanProgress progress{1, 1};
    bool favorite{false};
    std::string lastReadAt;
    std::string coverImageUrl;

    [[nodiscard]] double completion_percent() const {
        if (pageCount <= 0 || chapterCount <= 0) return 0.0;
        const int chapter = progress.chapter < 1 ? 1 : progress.chapter;
        const int page = progress.page < 1 ? 1 : progress.page;
        const double averagePagesPerChapter = static_cast<double>(pageCount) / ...
        static_cast<double>(chapterCount);
        double readPosition = (static_cast<double>(chapter - 1) * averagePagesPerChapter) + ...
        static_cast<double>(page);
        if (readPosition <= 1.0) return 0.0;
        double percent = readPosition / static_cast<double>(pageCount);
        if (percent < 0.0) return 0.0;
        if (percent > 1.0) return 1.0;
        return percent;
    }
};

struct ScanPageInfo {
    int chapter{0};
    int page{0};
    std::string mimeType;
    long long sizeBytes{0};
    std::string imageUrl;
};

struct ProfileSummary {
    std::string id;
    std::string displayName;
    std::string avatarColor{"#3b82f6"};
};

struct BookmarkSummary {
    long long id{0};
    std::string scanId;
    int chapter{1};
    int page{1};
    std::string note;
    std::string createdAt;
};

struct HistoryEntry {
    std::string scanId;
    std::string title;
    int chapter{1};
    int page{1};
    std::string readAt;
};

struct SearchResultSummary {
    std::string scanId;
    std::string title;
    int chapter{1};
    int page{1};
    std::string snippet;
    std::string source;
};

class ScanDataSource {
public:
    virtual ~ScanDataSource() = default;

    [[nodiscard]] virtual std::vector<ScanSummary> list_scans(const std::string& profile = ...
    "default") const = 0;
    [[nodiscard]] virtual std::vector<int> list_chapters(const std::string& scanId) const = 0;
    [[nodiscard]] virtual std::vector<ScanPageInfo> list_pages(const std::string& scanId, int ...
    chapter) const = 0;
    [[nodiscard]] virtual std::filesystem::path materialize_page(const std::string& scanId, ...
    ScanProgress progress) const = 0;
    [[nodiscard]] virtual ScanProgress load_progress(const std::string& scanId, const ...
    std::string& profile = "default") const = 0;
    virtual void save_progress(const std::string& scanId, ScanProgress progress, const ...
    std::string& profile = "default") const = 0;

    [[nodiscard]] virtual std::vector<ProfileSummary> list_profiles() const { return ...
    {ProfileSummary{"default", "Default", "#3b82f6"}}; }
    virtual void save_profile(const ProfileSummary&) const {}
    [[nodiscard]] virtual std::vector<std::string> list_favorites(const std::string&) const { ...
    return {}; }
    virtual void set_favorite(const std::string&, bool, const std::string& = "default") const {}
    [[nodiscard]] virtual std::vector<BookmarkSummary> list_bookmarks(const std::string&, const ...
    std::string& = "default") const { return {}; }
    virtual BookmarkSummary add_bookmark(const std::string& scanId, ScanProgress progress, ...
    const std::string& note, const std::string& profile = "default") const {
        (void)profile;
        return BookmarkSummary{0, scanId, progress.chapter, progress.page, note, ""};
    }
    [[nodiscard]] virtual std::vector<HistoryEntry> list_history(const std::string& = ...

```

```

"default", int = 20) const { return {}; }
[[nodiscard]] virtual std::vector<SearchResultSummary> search_text(const std::string&, int ...
= 20) const { return {}; }
};

```

5.2 2. Etat et widgets principaux de la fenetre GTK

La fenetre conserve la source active, les scans filtres, les options de lecture et les widgets de bibliotheque.

include/Main_Window.hpp - lignes 72-199

```

Gtk::Box m_VBox;
Gtk::MenuBar m_MenuBar;

Gtk::Grid m_Grid;
Gtk::Box m_LeftPanel;
Gtk::Box m_LibraryToolBar;
Gtk::ComboBoxText m_ModeCombo;
Gtk::ComboBoxText m_ProfileCombo;
Gtk::Entry m_ServerUrlEntry;
Gtk::SearchEntry m_SearchEntry;
Gtk::ComboBoxText m_SortCombo;
Gtk::CheckButton m_FavoritesOnly;
Gtk::Button m_ButtonRefresh;
Gtk::Button m_ButtonResume;
Gtk::Button m_ButtonOfflineSync;
Gtk::Button m_ButtonCreateProfile;
Gtk::Button m_ButtonHistory;
Gtk::Button m_ButtonSearchText;
Gtk::Button m_ButtonBookmarks;
Gtk::ProgressBar m_DownloadProgress;
Gtk::Label m_DownloadStatus;

std::vector<Gtk::Button*> m_ListButtonManga;
Gtk::Grid m_GridListeManga;
Gtk::ScrolledWindow m_ScrollListeManga;

Gtk::Grid m_GridImage_go_to;
Gtk::ScrolledWindow m_ScrollImage;
Gtk::Stack m_ReaderStack;
Gtk::Box m_PageHBox;
Gtk::Box m_WebtoonBox;
std::vector<Gtk::Widget*> m_WebtoonImages;

Gtk::Grid m_go_to;
Gtk::Entry m_Entry_page;
Gtk::Entry m_Entry_chapter;
Gtk::Button m_Button_go_to;
Gtk::Button m_ButtonFavorite;
Gtk::Button m_ButtonBookmark;
Gtk::Button m_ButtonShortcuts;
Gtk::ProgressBar m_ChapterProgress;
Gtk::Scale m_PageSlider;
Gtk::ScrolledWindow m_ThumbScroll;
Gtk::Box m_ThumbBox;
std::vector<Gtk::Widget*> m_ThumbButtons;

Gtk::MenuItem m_File;
Gtk::Menu m_FileMenu;
Gtk::MenuItem m_FileOpen;
Gtk::MenuItem m_FileSave;
Gtk::MenuItem m_FileQuit;

Gtk::MenuItem m_Download;
Gtk::Menu m_DownloadMenu;
Gtk::MenuItem m_UpdateScan;
Gtk::MenuItem m_UpdateAllScan;
Gtk::MenuItem m_NewScan;
Gtk::MenuItem m_OfflineSyncMenu;
Gtk::MenuItem m_PauseJobsMenu;
Gtk::MenuItem m_ResumeJobsMenu;
Gtk::MenuItem m_CancelJobsMenu;

Gtk::MenuItem m_View;
Gtk::Menu m_ViewMenu;
Gtk::CheckMenuItem m_DoublePageMenu;
Gtk::CheckMenuItem m_WebtoonMenu;
Gtk::MenuItem m_FullscreenMenu;

Gtk::MenuItem m_Help;
Gtk::Menu m_HelpMenu;
Gtk::MenuItem m_ShortcutsMenu;

Scan m_Scan;
Scan m_SecondScan;
Gtk::EventBox m_EventBox;
Gtk::EventBox m_SecondEventBox;

JsonScanRepository repository_;
CurlHttpClient httpClient_;
LelScansProvider provider_;

void on_menu_file_open();

```

```

void on_menu_file_save();
void on_menu_file_quit();
void on_menu_download_update();
void on_menu_download_new();
void update_all_scan();
void update_scan(const std::filesystem::path& folder);
void pause_download_queue();
void resume_download_queue();
void cancel_download_queue();

void configure_data_source();
[[nodiscard]] ScanDataSource& active_source();
[[nodiscard]] const ScanDataSource& active_source() const;
void on_mode_changed();
void on_profile_changed();
void create_profile_dialog();
void refresh_profile_list();
void refresh_scan_list();
[[nodiscard]] std::vector<ScanSummary> filtered_sorted_scans() const;
void open_scan(const std::filesystem::path& folder);
void open_scan_by_id(const std::string& scanId, ScanProgress progress = {1, 1});
void go_to();

void navigate_next();
void navigate_previous();
[[nodiscard]] std::optional<ScanProgress> next_progress() const;
[[nodiscard]] std::optional<ScanProgress> previous_progress() const;
[[nodiscard]] int current_page_index() const;
[[nodiscard]] int current_chapter_index() const;
void save_current_progress();
void render_current_page();
void render_webtoon_chapter();
void rebuild_thumbnails();
void refresh_progress_placeholders();
void on_page_slider_changed();
void toggle_favorite();
void add_bookmark();
void show_bookmarks();
void show_history();
void show_text_search();
void resume_last_reading();
void synchronize_offline();
void on_download_progress(const std::string& jobId, int current, int total, const ...
std::string& message);
void show_shortcuts();
void set_fullscreen_mode();
void set_double_page_mode(bool enabled);

```

5.3 3. Initialisation des callbacks de telechargement

La file de telechargement publie son etat vers la GUI sans bloquer le thread graphique.

src/Main_Window.cpp - lignes 108-126

```

Glib::signal_idle().connect([this, report]() {
    if (report.success) {
        m_DownloadProgress.set_fraction(1.0);
        m_DownloadStatus.set_text("Termine : " + report.id);
        refresh_scan_list();
        show_info("Tache terminee", "La tache " + report.id + " est terminee apres " + ...
std::to_string(report.attempts) + " tentative(s).");
    } else if (report.cancelled) {
        m_DownloadStatus.set_text("Annule : " + report.id);
        show_info("Tache annulee", "La tache " + report.id + " a ete annulee ...
proprement.");
    } else {
        m_DownloadStatus.set_text("Erreur : " + report.id);
        show_error("Tache en echec", "La tache " + report.id + " a echoue : " + ...
report.error);
    }
    return false;
});
}, [this](const std::string& jobId, int current, int total, const std::string& message) {
    on_download_progress(jobId, current, total, message);
});
downloadQueue->start();

```

5.4 4. Barre superieure : mode local, serveur, offline

Ces widgets donnent enfin un acces visible au mode serveur et au mode offline.

src/Main_Window.cpp - lignes 190-236

```

m_ModeCombo.append_text("Serveur local");
m_ModeCombo.append_text("Offline");
m_ModeCombo.set_active(0);
m_ModeCombo.signal_changed().connect(sigc::mem_fun(*this, &Main_Window::on_mode_changed));

```

```

m_ServerUrlEntry.set_text("http://127.0.0.1:8787");
m_ServerUrlEntry.set_placeholder_text("URL serveur");
m_SearchEntry.set_placeholder_text("Rechercher un scan");
m_SearchEntry.signal_search_changed().connect(sigc::mem_fun(*this, ...
&Main_Window::refresh_scan_list));
m_SortCombo.append_text("Titre");
m_SortCombo.append_text("Derniere lecture");
m_SortCombo.append_text("Avancement");
m_SortCombo.append_text("Chapitres");
m_SortCombo.append_text("Pages");
m_SortCombo.set_active(0);
m_SortCombo.signal_changed().connect(sigc::mem_fun(*this, &Main_Window::refresh_scan_list));
m_ProfileCombo.signal_changed().connect(sigc::mem_fun(*this, ...
&Main_Window::on_profile_changed));
m_FavoritesOnly.signal_toggled().connect(sigc::mem_fun(*this, ...
&Main_Window::refresh_scan_list));
m_ButtonRefresh.signal_clicked().connect(sigc::mem_fun(*this, ...
&Main_Window::refresh_scan_list));
m_ButtonResume.signal_clicked().connect(sigc::mem_fun(*this, ...
&Main_Window::resume_last_reading));
m_ButtonOfflineSync.signal_clicked().connect(sigc::mem_fun(*this, ...
&Main_Window::synchronize_offline));
m_ButtonCreateProfile.signal_clicked().connect(sigc::mem_fun(*this, ...
&Main_Window::create_profile_dialog));
m_ButtonHistory.signal_clicked().connect(sigc::mem_fun(*this, &Main_Window::show_history));
m_ButtonSearchText.signal_clicked().connect(sigc::mem_fun(*this, ...
&Main_Window::show_text_search));
m_ButtonBookmarks.signal_clicked().connect(sigc::mem_fun(*this, ...
&Main_Window::show_bookmarks));
m_DownloadProgress.set_show_text(true);
m_DownloadProgress.set_text("0 %");

m_LibraryToolbar.set_spacing(6);
m_LibraryToolbar.pack_start(m_ModeCombo, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ServerUrlEntry, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ProfileCombo, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_SearchEntry, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_SortCombo, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_FavoritesOnly, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonRefresh, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonResume, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonOfflineSync, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonCreateProfile, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonHistory, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonSearchText, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_ButtonBookmarks, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_DownloadProgress, Gtk::PACK_SHRINK);
m_LibraryToolbar.pack_start(m_DownloadStatus, Gtk::PACK_SHRINK);

m_ScrollListeManga.set_size_request(360, HEIGHT);
m_ScrollListeManga.set_policy(Gtk::POLICY_AUTOMATIC, Gtk::POLICY_AUTOMATIC);

```

5.5 5. Choix dynamique de la source active

La GUI instancie FileSystemScanDataSource ou ApiScanDataSource selon le choix utilisateur.

src/Main_Window.cpp - lignes 301-315

```

const Glib::ustring modeText = m_ModeCombo.get_active_text();
if (modeText == "Serveur local") {
    mode_ = DataMode::Server;
    dataSource_ = std::make_unique<ApiScanDataSource>(m_ServerUrlEntry.get_text().raw(), ...
"cache/api");
} else if (modeText == "Offline") {
    mode_ = DataMode::Offline;
    dataSource_ = std::make_unique<FileSystemScanDataSource>("scan");
} else {
    mode_ = DataMode::Local;
    dataSource_ = std::make_unique<FileSystemScanDataSource>("scan");
}
refresh_profile_list();
refresh_scan_list();
}

```

5.6 6. Bibliotheque : recherche, tri, favoris, cartes

La bibliotheque reconstruit des cartes de scan avec avancement, favoris et bouton reprendre.

src/Main_Window.cpp - lignes 383-461

```

auto scans = active_source().list_scans(currentProfile_);
const std::string query = lower_ascii(m_SearchEntry.get_text().raw());
const bool favoritesOnly = m_FavoritesOnly.get_active();
scans.erase(std::remove_if(scans.begin(), scans.end(), [&](const ScanSummary& scan) {
    if (favoritesOnly && !scan.favorite) return true;
    if (!query.empty() && lower_ascii(scan.title).find(query) == std::string::npos && ...

```



```

lower_ascii(scan.id).find(query) == std::string::npos) return true;
return false;
}), scans.end());

const std::string sort = m_SortCombo.get_active_text().raw();
if (sort == "Derniere lecture") {
    std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.lastReadAt, a.title) > std::tie(b.lastReadAt, b.title); });
} else if (sort == "Avancement") {
    std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.progress.chapter, a.progress.page, a.title) > std::tie(b.progress.chapter, ...
b.progress.page, b.title); });
} else if (sort == "Chapitres") {
    std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.chapterCount, a.title) > std::tie(b.chapterCount, b.title); });
} else if (sort == "Pages") {
    std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.pageCount, a.title) > std::tie(b.pageCount, b.title); });
} else {
    std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return a.title ...
< b.title; });
}
return scans;
}

void Main_Window::refresh_scan_list() {
    for (auto* button : m_ListButtonManga) m_GridListeManga.remove(*button);
    m_ListButtonManga.clear();

    std::vector<ScanSummary> scans;
    try { scans = filtered_sorted_scans(); } catch (const std::exception& error) { ...
show_error("Bibliothèque indisponible", error.what()); return; }

    int row = 0;
    int col = 0;
    constexpr int columns = 2;
    for (const auto& scan : scans) {
        auto* button = Gtk::make_managed<Gtk::Button>();
        auto* card = Gtk::make_managed<Gtk::Box>(Gtk::ORIENTATION_VERTICAL);
        card->set_spacing(4);
        card->set_margin_top(6);
        card->set_margin_bottom(6);
        card->set_margin_left(6);
        card->set_margin_right(6);

        try {
            auto chapters = active_source().list_chapters(scan.id);
            if (!chapters.empty()) {
                auto pages = active_source().list_pages(scan.id, chapters.front());
                if (!pages.empty()) {
                    const auto coverPath = active_source().materialize_page(scan.id, ...
ScanProgress(pages.front().chapter, pages.front().page));
                    auto pixbuf = Gdk::Pixbuf::create_from_file(coverPath.string(), 120, 160, ...
true);
                    auto* image = Gtk::make_managed<Gtk::Image>(pixbuf);
                    card->pack_start(*image, Gtk::PACK_SHRINK);
                }
            }
        } catch (...) {
            auto* noCover = Gtk::make_managed<Gtk::Label>("Pas de couverture");
            card->pack_start(*noCover, Gtk::PACK_SHRINK);
        }

        std::ostringstream label;
        label << (scan.favorite ? "*" : "") << scan.title << "\n"
        << scan.chapterCount << " chap. - " << scan.pageCount << " pages\n"
        << "Progression : ch. " << scan.progress.chapter << ", p. " << scan.progress.page ...
<< "\n"
        << percent_label(scan);
        if (!scan.lastReadAt.empty()) label << "\nDerniere lecture : " << scan.lastReadAt;
        label << "\nClicquer pour reprendre";

        auto* text = Gtk::make_managed<Gtk::Label>(label.str());
        text->set_line_wrap(true);
        text->set_justify(Gtk::JUSTIFY_CENTER);
        card->pack_start(*text, Gtk::PACK_SHRINK);

        auto* progress = Gtk::make_managed<Gtk::ProgressBar>();
        progress->set_show_text(true);
        const double completion = scan.completion_percent();
        progress->set_fraction(std::clamp(completion, 0.0, 1.0));
        progress->set_text(completion <= 0.0 ? "0 %" : "en cours");
        card->pack_start(*progress, Gtk::PACK_SHRINK);
    }
}

```

5.7 7. Lecture : page simple, double page, webtoon, zoom

Le lecteur gere plusieurs modes d'affichage tout en gardant une source unique de pages.

src/Main_Window.cpp - lignes 540-609

```

if (currentScanId.empty()) return;
currentPages_ = active_source().list_pages(currentScanId_, currentProgress_.chapter);
if (currentPages_.empty()) throw std::runtime_error("Chapitre vide ou introuvable.");
if (std::none_of(currentPages_.begin(), currentPages_.end(), [&](const auto& p) { return ...
p.page == currentProgress_.page; })) currentProgress_.page = currentPages_.front().page;

```

```

    if (webtoonMode_) {
        render_webtoon_chapter();
    } else {
        m_ReaderStack.set_visible_child("paged");
        const auto pagePath = active_source().materialize_page(currentScanId_, ...
currentProgress_);
        m_Scan.set_page(pagePath);
        if (doublePageMode_) {
            if (auto second = next_progress()) {
                try { m_SecondScan.set_page(active_source().materialize_page(currentScanId_, ...
*second)); m_SecondEventBox.show(); }
                catch (...) { m_SecondEventBox.hide(); }
            } else {
                m_SecondEventBox.hide();
            }
        } else {
            m_SecondEventBox.hide();
        }
    }
}

save_current_progress();
rebuild_thumbnails();
refresh_progress_placeholders();
}

void Main_Window::render_webtoon_chapter() {
    for (auto* widget : m_WebtoonImages) m_WebtoonBox.remove(*widget);
    m_WebtoonImages.clear();
    for (const auto& page : currentPages_) {
        try {
            auto path = active_source().materialize_page(currentScanId_, ...
ScanProgress{page.chapter, page.page});
            auto* image = Gtk::make_managed<Gtk::Image>(path.string());
            image->set_margin_bottom(12);
            m_WebtoonBox.pack_start(*image, Gtk::PACK_SHRINK);
            m_WebtoonImages.push_back(image);
        } catch (...) {}
    }
    m_ReaderStack.set_visible_child("webtoon");
    m_WebtoonBox.show_all();
}

void Main_Window::rebuild_thumbnails() {
    for (auto* widget : m_ThumbButtons) m_ThumbBox.remove(*widget);
    m_ThumbButtons.clear();
    for (const auto& page : currentPages_) {
        auto* button = Gtk::make_managed<Gtk::Button>();
        auto* box = Gtk::make_managed<Gtk::Box>(Gtk::ORIENTATION_VERTICAL);
        try {
            const auto path = active_source().materialize_page(currentScanId_, ...
ScanProgress{page.chapter, page.page});
            auto pixbuf = Gdk::Pixbuf::create_from_file(path.string(), 56, 74, true);
            auto* image = Gtk::make_managed<Gtk::Image>(pixbuf);
            box->pack_start(*image, Gtk::PACK_SHRINK);
        } catch (...) {}
        auto* label = Gtk::make_managed<Gtk::Label>(std::to_string(page.page));
        box->pack_start(*label, Gtk::PACK_SHRINK);
        button->add(*box);
        button->signal_clicked().connect([this, page]() { currentProgress_ = {page.chapter, ...
page.page}; render_current_page(); });
        m_ThumbBox.pack_start(*button, Gtk::PACK_SHRINK);
        m_ThumbButtons.push_back(button);
    }
    m_ThumbBox.show_all();
}

void Main_Window::save_current_progress() {
    if (currentScanId_.empty()) return;
    try { active_source().save_progress(currentScanId_, currentProgress_, currentProfile_); } ...
    catch (const std::exception& error) { std::cerr << "Sauvegarde impossible: " << ...
    error.what() << std::endl; }
}

```

5.8 8. Marque-pages, historique et recherche OCR dans GTK

Ces dialogues consomment les methodes ajoutes au contrat ScanDataSource.

src/Main_Window.cpp - lignes 649-738

```

if (currentScanId_.empty()) { show_error("Aucun scan ouvert", "Ouvre un scan avant de ...
consulter les marque-pages."); return; }
try {
    const auto bookmarks = active_source().list_bookmarks(currentScanId_, currentProfile_);
    Gtk::Dialog dialog("Marque-pages", *this, true);
    dialog.add_button("Fermer", Gtk::RESPONSE_CLOSE);
    auto* area = dialog.get_content_area();
    if (bookmarks.empty()) {
        auto* label = Gtk::make_managed<Gtk::Label>("Aucun marque-page pour ce scan.");
        area->pack_start(*label, Gtk::PACK_SHRINK);
    }
    for (const auto& bookmark : bookmarks) {
        std::ostringstream text;
        text << "Chapitre " << bookmark.chapter << ", page " << bookmark.page;
        if (!bookmark.note.empty()) text << " - " << bookmark.note;
    }
}

```

```

        auto* button = Gtk::make_managed<Gtk::Button>(text.str());
        button->signal_clicked().connect([this, &dialog, bookmark]() {
            currentProgress = ScanProgress{bookmark.chapter, bookmark.page};
            dialog.response(Gtk::RESPONSE_OK);
        });
        area->pack_start(*button, Gtk::PACK_SHRINK);
    }
    dialog.show_all_children();
    if (dialog.run() == Gtk::RESPONSE_OK) render_current_page();
} catch (const std::exception& error) { show_error("Marque-pages indisponibles", ...
error.what()); }
}

void Main_Window::show_history() {
    try {
        const auto history = active_source().list_history(currentProfile_, 20);
        Gtk::Dialog dialog("Historique de lecture", *this, true);
        dialog.add_button("Fermer", Gtk::RESPONSE_CLOSE);
        auto* area = dialog.get_content_area();
        if (history.empty()) {
            auto* label = Gtk::make_managed<Gtk::Label>("Aucun historique pour ce profil.");
            area->pack_start(*label, Gtk::PACK_SHRINK);
        }
        for (const auto& item : history) {
            std::ostringstream text;
            text << item.title << " - ch. " << item.chapter << ", p. " << item.page;
            if (!item.readAt.empty()) text << " - " << item.readAt;
            auto* button = Gtk::make_managed<Gtk::Button>(text.str());
            button->signal_clicked().connect([this, &dialog, item]() {
                open_scan_by_id(item.scanId, ScanProgress{item.chapter, item.page});
                dialog.response(Gtk::RESPONSE_OK);
            });
            area->pack_start(*button, Gtk::PACK_SHRINK);
        }
        dialog.show_all_children();
        (void)dialog.run();
    } catch (const std::exception& error) { show_error("Historique indisponible", ...
error.what()); }
}

void Main_Window::show_text_search() {
    Gtk::Dialog input("Recherche OCR / texte", *this, true);
    input.add_button("Annuler", Gtk::RESPONSE_CANCEL);
    input.add_button("Rechercher", Gtk::RESPONSE_OK);
    Gtk::Entry queryEntry;
    queryEntry.set_placeholder_text("Mot ou expression a chercher dans les pages OCR");
    queryEntry.set_text(m_SearchEntry.get_text());
    input.get_content_area()->pack_start(queryEntry);
    input.show_all_children();
    if (input.run() != Gtk::RESPONSE_OK) return;
    const std::string query = queryEntry.get_text().raw();
    if (query.empty()) return;

    try {
        const auto results = active_source().search_text(query, 30);
        Gtk::Dialog dialog("Resultats OCR", *this, true);
        dialog.add_button("Fermer", Gtk::RESPONSE_CLOSE);
        auto* area = dialog.get_content_area();
        if (results.empty()) {
            auto* label = Gtk::make_managed<Gtk::Label>("Aucun resultat indexe. Lance ...
/api/admin/ocr ou ajoute des fichiers .txt sidecar.");
            label->set_line_wrap(true);
            area->pack_start(*label, Gtk::PACK_SHRINK);
        }
        for (const auto& result : results) {
            std::ostringstream text;
            text << result.title << " - ch. " << result.chapter << ", p. " << result.page << ...
"\n" << result.snippet;
            auto* button = Gtk::make_managed<Gtk::Button>(text.str());
            button->signal_clicked().connect([this, &dialog, result]() {
                open_scan_by_id(result.scanId, ScanProgress{result.chapter, result.page});
                dialog.response(Gtk::RESPONSE_OK);
            });
            area->pack_start(*button, Gtk::PACK_SHRINK);
        }
        dialog.show_all_children();
        (void)dialog.run();
    } catch (const std::exception& error) { show_error("Recherche impossible", error.what()); }
}

```

5.9 9. Synchronisation offline depuis le serveur

Le bouton offline lance un job de synchronisation via DownloadQueue et OfflineLibrarySync.

src/Main_Window.cpp - lignes 740-778

```

Glib::signal_idle().connect([this, jobId, current, total, message]() {
    if (total > 0) {
        const double fraction = std::clamp(static_cast<double>(current) / ...
static_cast<double>(total), 0.0, 1.0);
        m_DownloadProgress.set_fraction(fraction);
        m_DownloadProgress.set_text(std::to_string(static_cast<int>(std::round(fraction * ...
100.0))) + " %");
    } else {
        m_DownloadProgress.pulse();
    }
});

```

```

        m_DownloadProgress.set_text("en cours");
    }
    m_DownloadStatus.set_text(jobId + " : " + message);
    return false;
});
}

void Main_Window::resume_last_reading() {
    try {
        const auto history = active_source().list_history(currentProfile_, 1);
        if (!history.empty()) { open_scan_by_id(history.front().scanId, ...
ScanProgress(history.front().chapter, history.front().page)); return; }
        const auto scans = active_source().list_scans(currentProfile_);
        if (!scans.empty()) open_scan_by_id(scans.front().id, scans.front().progress);
    } catch (const std::exception& error) { show_error("Reprise impossible", error.what()); }
}

void Main_Window::synchronize_offline() {
    const std::string baseUrl = m_ServerUrlEntry.get_text().raw();
    const std::string profile = currentProfile_;
    downloadQueue->push(DownloadJob{"offline-sync", [baseUrl, profile](DownloadJobContext& ...
ctx) {
        ApiScanDataSource api(baseUrl, "cache/api");
        OfflineLibrarySync sync(api, "scan");
        auto report = sync.sync_all(profile, [&](const OfflineSyncProgress& progress) {
            ctx.report(progress.downloadedPages + progress.skippedPages + progress.failedPages, ...
0,
            "scan=" + progress.scanId + " ch=" + std::to_string(progress.chapter) + ...
" p=" + std::to_string(progress.page));
            ctx.wait_if_paused();
            if (ctx.cancelled()) throw std::runtime_error("offline sync cancelled");
        });
        ctx.report(report.downloadedPages, report.downloadedPages + report.skippedPages + ...
report.failedPages, "synchronisation offline terminee");
    }, 1));
    show_info("Synchronisation lancee", "Le telechargement offline est place dans la file de ...
taches. Les fichiers seront copies dans ./scan.");
}

```

5.10 10. File de jobs asynchrones

DownloadQueue expose pause, reprise, annulation, progression et retry sans dependance GTK.

include/application/DownloadQueue.hpp - lignes 13-93

```

/**
 * @brief Contexte fourni a une tache de telechargement.
 *
 * La tache peut tester 'cancelled()' entre deux operations longues et publier une progression.
 * L'annulation reste cooperative : la file ne tue jamais brutalement un thread en cours.
 */
struct DownloadJobContext {
    std::atomic<bool>& cancelRequested;
    std::function<bool()> pauseRequested;
    std::function<void()> waitWhilePaused;
    std::function<void(int current, int total, const std::string& message)> progress;

    [[nodiscard]] bool cancelled() const { return cancelRequested.load(); }
    [[nodiscard]] bool paused() const { return pauseRequested ? pauseRequested() : false; }
    void wait_if_paused() const {
        if (waitWhilePaused) waitWhilePaused();
    }
    void report(int current, int total, const std::string& message) const {
        if (progress) progress(current, total, message);
    }
};

struct DownloadJob {
    std::string id;
    std::function<void(DownloadJobContext&)> work;
    int maxRetries{2};
};

struct DownloadJobReport {
    std::string id;
    bool success{false};
    bool cancelled{false};
    int attempts{0};
    std::string error;
};

/**
 * @brief Petite file de travaux en arriere-plan pour synchronisations, mises a jour et OCR.
 *
 * La file ne connait pas le contenu des travaux : elle execute des callbacks. Cette separation
 * permet de brancher la synchronisation offline, la mise a jour de scans ou une indexation OCR
 * sans coupler la GUI aux details reseau ou disque.
 */
class DownloadQueue {
public:
    using CompletionCallback = std::function<void(const DownloadJobReport&)>;
    using ProgressCallback = std::function<void(const std::string& jobId, int current, int ...
total, const std::string& message)>;

```

```

explicit DownloadQueue(CompletionCallback onComplete = {}, ProgressCallback onProgress = ...
{});
DownloadQueue();

void start();
void stop();
void cancel();
void pause();
void resume();
void clear();
void push(DownloadJob job);

[[nodiscard]] bool running() const;
[[nodiscard]] bool paused() const;
[[nodiscard]] bool cancellation_requested() const;
[[nodiscard]] std::size_t pending_jobs() const;

private:
CompletionCallback onComplete_;
ProgressCallback onProgress_;
std::queue<DownloadJob> jobs_;
mutable std::mutex mutex_;
std::condition_variable condition_;
std::thread worker_;
std::atomic<bool> running_{false};
std::atomic<bool> stopRequested_{false};
std::atomic<bool> cancelRequested_{false};
std::atomic<bool> pauseRequested_{false};

void loop();
};

#endif

```

5.11 11. Execution thread-safe des jobs

La boucle worker attend la reprise, respecte l'annulation et remonte un rapport de fin.

src/application/DownloadQueue.cpp - lignes 36-115

```

void DownloadQueue::resume() {
    pauseRequested_ = false;
    condition_.notify_all();
}

void DownloadQueue::clear() {
    std::lock_guard<std::mutex> lock(mutex_);
    std::queue<DownloadJob> empty;
    std::swap(jobs_, empty);
}

void DownloadQueue::push(DownloadJob job) {
    {
        std::lock_guard<std::mutex> lock(mutex_);
        jobs_.push(std::move(job));
    }
    cancelRequested_ = false;
    condition_.notify_one();
}

bool DownloadQueue::running() const { return running_; }
bool DownloadQueue::paused() const { return pauseRequested_; }
bool DownloadQueue::cancellation_requested() const { return cancelRequested_; }

std::size_t DownloadQueue::pending_jobs() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return jobs_.size();
}

void DownloadQueue::loop() {
    while (!stopRequested_) {
        DownloadJob job;
        {
            std::unique_lock<std::mutex> lock(mutex_);
            condition_.wait(lock, [this] {
                return stopRequested_ || (!pauseRequested_ && !jobs_.empty());
            });
            if (stopRequested_) break;
            if (jobs_.empty()) continue;
            job = std::move(jobs_.front());
            jobs_.pop();
        }

        DownloadJobReport report;
        report.id = job.id;
        for (int attempt = 1; attempt <= job.maxRetries + 1; ++attempt) {
            report.attempts = attempt;
            report.cancelled = false;
            try {
                if (!job.work) throw std::runtime_error("empty job");
                DownloadJobContext context{
                    cancelRequested_,
                    [this] () { return pauseRequested_.load(); },
                    [this] () {
                        std::unique_lock<std::mutex> lock(mutex_);

```

```

        condition_.wait(lock, [this] {
            return stopRequested_.load() || cancelRequested_.load() || ...
        });
        !pauseRequested_.load();
    },
    [this, jobId = job.id](int current, int total, const std::string& message) {
        if (onProgress_) onProgress_(jobId, current, total, message);
    }
};
if (context.cancelled()) {
    report.cancelled = true;
    report.error = "cancelled before start";
    break;
}
context.report(0, 0, "demarrage");
job.work(context);
if (context.cancelled()) {
    report.cancelled = true;
    report.error = "cancelled";
    break;
}
report.success = true;
report.error.clear();
break;
} catch (const std::exception& error) {
    report.error = error.what();
}

```

5.12 12. Copie offline complete d'une bibliotheque

OfflineLibrarySync parcourt les scans, chapitres et pages d'une source, puis les materialise sur le disque.

src/application/OfflineLibrarySync.cpp - lignes 1-86

```

#include "application/OfflineLibrarySync.hpp"
#include "infrastructure/JsonScanRepository.hpp"

#include <algorithm>
#include <cctype>
#include <filesystem>
#include <utility>

OfflineLibrarySync::OfflineLibrarySync(const ScanDataSource& source, std::filesystem::path ...
    outputRoot)
    : source_(source), outputRoot_(std::move(outputRoot)) {}

OfflineSyncReport OfflineLibrarySync::sync_all(const std::string& profile, ProgressCallback ...
    callback) const {
    OfflineSyncReport total;
    for (const auto& scan : source_.list_scans(profile)) {
        auto report = sync_scan(scan.id, profile, callback);
        total.scans += report.scans;
        total.chapters += report.chapters;
        total.downloadedPages += report.downloadedPages;
        total.skippedPages += report.skippedPages;
        total.failedPages += report.failedPages;
    }
    return total;
}

OfflineSyncReport OfflineLibrarySync::sync_scan(
    const std::string& scanId,
    const std::string& profile,
    ProgressCallback callback) const {
    OfflineSyncReport report;
    OfflineSyncProgress progress;
    progress.scanId = scanId;

    const auto chapters = source_.list_chapters(scanId);
    report.scans = 1;
    report.chapters = static_cast<int>(chapters.size());

    for (int chapter : chapters) {
        for (const auto& page : source_.list_pages(scanId, chapter)) {
            progress.chapter = page.chapter;
            progress.page = page.page;

            try {
                const auto sourcePath = source_.materialize_page(scanId, ...
                    ScanProgress{page.chapter, page.page});
                const auto destination = outputRoot_ / scanId / std::to_string(page.chapter) /
                    (std::to_string(page.page) + extension_from_path(sourcePath));

                std::filesystem::create_directories(destination.parent_path());
                if (std::filesystem::exists(destination) &&
                    std::filesystem::file_size(destination) == ...
                    std::filesystem::file_size(sourcePath)) {
                    ++report.skippedPages;
                    ++progress.skippedPages;
                } else {
                    std::filesystem::copy_file(sourcePath, destination, ...
                        std::filesystem::copy_options::overwrite_existing);
                    ++report.downloadedPages;
                    ++progress.downloadedPages;
                }
            } catch (...) {
                ++report.failedPages;
                ++progress.failedPages;
            }
        }
    }
}

```

```

    }
    } catch (...) {
        ++report.failedPages;
        ++progress.failedPages;
    }

    if (callback) callback(progress);
}

}

try {
    JsonScanRepository repository;
    repository.save_progress(outputRoot_ / scanId, source_.load_progress(scanId, profile));
} catch (...) {
    // La copie des images est prioritaire. Une progression illisible ne doit pas annuler ...
    la synchro.
}

return report;
}

std::string OfflineLibrarySync::extension_from_path(const std::filesystem::path& path) {
    std::string extension = path.extension().string();
    std::transform(extension.begin(), extension.end(), extension.begin(), [](unsigned char c) {
        return static_cast<char>(std::tolower(c));
    });

    if (extension == ".jpg" || extension == ".jpeg" || extension == ".png" || extension == ...
        ".webp") {
        return extension;
    }
    return ".jpg";
}

```

5.13 13. Source locale : bibliotheque, profils et recherche texte

La source locale lit les dossiers scan/, stocke les donnees de profil dans cache/profiles et cherche dans les fichiers .txt adjacents.

src/application/FileSystemScanDataSource.cpp - lignes 92-288

```

std::vector<ScanSummary> FileSystemScanDataSource::list_scans(const std::string& profile) const ...
{
    std::vector<ScanSummary> scans;
    std::set<std::string> favorites;
    for (const auto& line : read_lines(profile_file(profile, "favorites.txt"))) ...
        favorites.insert(line);

    if (!std::filesystem::exists(scanRoot_)) return scans;
    for (const auto& scanEntry : std::filesystem::directory_iterator(scanRoot_)) {
        if (!scanEntry.is_directory()) continue;
        const std::string id = scanEntry.path().filename().string();
        int chapterCount = 0;
        int pageCount = 0;
        std::string cover;
        for (const auto& chapterEntry : std::filesystem::directory_iterator(scanEntry.path())) {
            if (!chapterEntry.is_directory() || ...
                parse_positive_int(chapterEntry.path().filename().string()) == 0) continue;
            ++chapterCount;
            for (const auto& pageEntry : ...
                std::filesystem::directory_iterator(chapterEntry.path())) {
                if (pageEntry.is_regular_file() && is_image_file(pageEntry.path())) {
                    ++pageCount;
                    if (cover.empty()) cover = pageEntry.path().string();
                }
            }
        }
        ScanSummary summary{id, title_from_id(id), chapterCount, pageCount, ScanProgress{1, 1}, ...
            false, "", ""};
        summary.progress = load_progress(id, profile);
        summary.favorite = favorites.count(id) > 0;
        summary.coverImageUrl = cover;
        scans.push_back(summary);
    }
    std::sort(scans.begin(), scans.end(), [](const auto& left, const auto& right) { return ...
        left.title < right.title; });
    return scans;
}

std::vector<int> FileSystemScanDataSource::list_chapters(const std::string& scanId) const {
    std::vector<int> chapters;
    const auto scanPath = scanRoot_ / scanId;
    if (!std::filesystem::exists(scanPath)) return chapters;
    for (const auto& entry : std::filesystem::directory_iterator(scanPath)) {
        if (entry.is_directory()) {
            const int chapter = parse_positive_int(entry.path().filename().string());
            if (chapter > 0) chapters.push_back(chapter);
        }
    }
    std::sort(chapters.begin(), chapters.end());
    return chapters;
}

std::vector<ScanPageInfo> FileSystemScanDataSource::list_pages(const std::string& scanId, int ...

```

```

chapter) const {
    std::vector<ScanPageInfo> pages;
    const auto chapterPath = scanRoot_ / scanId / std::to_string(chapter);
    if (!std::filesystem::exists(chapterPath)) return pages;
    for (const auto& entry : std::filesystem::directory_iterator(chapterPath)) {
        if (!entry.is_regular_file() || !is_image_file(entry.path())) continue;
        const int page = parse_positive_int(entry.path().stem().string());
        if (page > 0) pages.push_back({chapter, page, "", static_cast<long ...
long>(std::filesystem::file_size(entry.path())), entry.path().string()});
    }
    std::sort(pages.begin(), pages.end(), [](const auto& left, const auto& right) { return ...
left.page < right.page; });
    return pages;
}

std::filesystem::path FileSystemScanDataSource::materialize_page(const std::string& scanId, ...
ScanProgress progress) const {
    progress.normalize();
    const auto chapterPath = scanRoot_ / scanId / std::to_string(progress.chapter);
    static const char* extensions[] = {".jpg", ".jpeg", ".png", ".webp"};
    for (const auto* extension : extensions) {
        auto candidate = chapterPath / (std::to_string(progress.page) + extension);
        if (std::filesystem::exists(candidate) && std::filesystem::is_regular_file(candidate)) ...
return candidate;
    }
    throw std::runtime_error("Page introuvable dans la source fichier: " + scanId);
}

ScanProgress FileSystemScanDataSource::load_progress(const std::string& scanId, const ...
std::string& profile) const {
    if (profile != "default") {
        const auto file = profile_root(profile) / "progress" / (safe_token(scanId) + ".json");
        std::ifstream in(file);
        if (in) {
            std::ostringstream buffer; buffer << in.rdbuf();
            ScanProgress progress{scangui::json::int_field(buffer.str(), "chapter", 1), ...
scangui::json::int_field(buffer.str(), "page", 1)};
            progress.normalize();
            return progress;
        }
    }
    if (auto metadata = repository_.load(scanRoot_ / scanId)) return metadata->saveProgress;
    return {1, 1};
}

void FileSystemScanDataSource::save_progress(const std::string& scanId, ScanProgress progress, ...
const std::string& profile) const {
    progress.normalize();
    if (profile == "default") {
        repository_.save_progress(scanRoot_ / scanId, progress);
    }
    const auto file = profile_root(profile) / "progress" / (safe_token(scanId) + ".json");
    std::filesystem::create_directories(file.parent_path());
    std::ofstream out(file, std::ios::trunc);
    out << "{ \"scanId\": \"" << scanId << "\", \"chapter\": " << progress.chapter << "...
progress.page << ", \"page\": " << progress.page << "}" << "\n";

    auto history = read_lines(profile_file(profile, "history.txt"));
    history.insert(history.begin(), scanId + "|" + std::to_string(progress.chapter) + "|" + ...
std::to_string(progress.page) + "|local");
    if (history.size() > 100) history.resize(100);
    write_lines(profile_file(profile, "history.txt"), history);
}

std::vector<ProfileSummary> FileSystemScanDataSource::list_profiles() const {
    std::vector<ProfileSummary> profiles{{"default", "Default", "#3b82f6"}};
    const auto root = std::filesystem::path("cache") / "profiles";
    if (!std::filesystem::exists(root)) return profiles;
    for (const auto& entry : std::filesystem::directory_iterator(root)) {
        if (!entry.is_directory()) continue;
        const std::string id = entry.path().filename().string();
        if (id != "default") profiles.push_back({id, title_from_id(id), "#64748b"});
    }
    return profiles;
}

void FileSystemScanDataSource::save_profile(const ProfileSummary& profile) const {
    std::filesystem::create_directories(profile_root(profile.id));
    std::ofstream out(profile_file(profile.id, "profile.json"), std::ios::trunc);
    out << "{ \"id\": \"" << profile.id << "\", \"displayName\": \"" << ...
scangui::json::escape(profile.displayName) << "\", \"avatarColor\": \"" << ...
scangui::json::escape(profile.avatarColor) << "\"}" << "\n";
}

std::vector<std::string> FileSystemScanDataSource::list_favorites(const std::string& profile) ...
const {
    return read_lines(profile_file(profile, "favorites.txt"));
}

void FileSystemScanDataSource::set_favorite(const std::string& scanId, bool favorite, const ...
std::string& profile) const {
    auto lines = read_lines(profile_file(profile, "favorites.txt"));
    lines.erase(std::remove(lines.begin(), lines.end(), scanId), lines.end());
    if (favorite) lines.push_back(scanId);
    write_lines(profile_file(profile, "favorites.txt"), lines);
}

std::vector<BookmarkSummary> FileSystemScanDataSource::list_bookmarks(const std::string& ...
scanId, const std::string& profile) const {
    std::vector<BookmarkSummary> bookmarks;
    long long id = 1;
    for (const auto& line : read_lines(profile_file(profile, "bookmarks.txt"))) {
        auto parts = split_pipe(line);

```



```

        if (parts.size() < 4 || parts[0] != scanId) continue;
        bookmarks.push_back({id++, parts[0], parse_positive_int(parts[1]), ...
        parse_positive_int(parts[2]), parts[3], "local"});
    }
    return bookmarks;
}

BookmarkSummary FileSystemScanDataSource::add_bookmark(const std::string& scanId, ScanProgress ...
progress, const std::string& note, const std::string& profile) const {
    progress.normalize();
    auto lines = read_lines(profile_file(profile, "bookmarks.txt"));
    lines.push_back(scanId + "|" + std::to_string(progress.chapter) + "|" + ...
    std::to_string(progress.page) + "|" + note);
    write_lines(profile_file(profile, "bookmarks.txt"), lines);
    return {static_cast<long long>(lines.size()), scanId, progress.chapter, progress.page, ...
    note, "local"};
}

std::vector<HistoryEntry> FileSystemScanDataSource::list_history(const std::string& profile, ...
int limit) const {
    std::vector<HistoryEntry> history;
    for (const auto& line : read_lines(profile_file(profile, "history.txt"))) {
        auto parts = split_pipe(line);
        if (parts.size() < 4) continue;
        history.push_back({parts[0], title_from_id(parts[0]), parse_positive_int(parts[1]), ...
        parse_positive_int(parts[2]), parts[3]});
        if (static_cast<int>(history.size()) >= limit) break;
    }
    return history;
}

std::vector<SearchResultSummary> FileSystemScanDataSource::search_text(const std::string& ...
query, int limit) const {
    std::vector<SearchResultSummary> results;
    if (query.empty() || limit <= 0 || !std::filesystem::exists(scanRoot_)) return results;
    std::string q = query;
    std::transform(q.begin(), q.end(), q.begin(), [](unsigned char c) { return ...
    static_cast<char>(std::tolower(c)); });

    for (const auto& scanEntry : std::filesystem::directory_iterator(scanRoot_)) {
        if (!scanEntry.is_directory()) continue;
        const std::string scanId = scanEntry.path().filename().string();
        for (const auto& entry : ...
        std::filesystem::recursive_directory_iterator(scanEntry.path())) {
            if (!entry.is_regular_file() || entry.path().extension() != ".txt") continue;
            std::ifstream in(entry.path());
            std::ostringstream buffer;
            buffer << in.rdbuf();
            std::string text = buffer.str();
            std::string lower = text;
            std::transform(lower.begin(), lower.end(), lower.begin(), [](unsigned char c) { ...
            return static_cast<char>(std::tolower(c)); });
            auto pos = lower.find(q);
            if (pos == std::string::npos) continue;

            int chapter = 1;
            int page = 1;
            auto parentName = entry.path().parent_path().filename().string();
            auto stemName = entry.path().stem().string();
            chapter = parse_positive_int(parentName);
            page = parse_positive_int(stemName);
            if (chapter <= 0) chapter = 1;
            if (page <= 0) page = 1;

            const std::size_t begin = pos > 80 ? pos - 80 : 0;
            std::string snippet = text.substr(begin, 220);
            results.push_back({scanId, title_from_id(scanId), chapter, page, snippet, "local"});
            if (static_cast<int>(results.size()) >= limit) return results;
        }
    }
    return results;
}

```

5.14 14. Source API : scans, images, progression, OCR

La source API convertit les appels HTTP du serveur en objets métier et met les images en cache local.

src/application/ApiScanDataSource.cpp - lignes 49-181

```

std::vector<ScanSummary> ApiScanDataSource::list_scans(const std::string& profile) const {
    const std::string json = http::get_text(endpoint("/api/scans?profile=" + ...
    url_encode(profile)));
    std::vector<ScanSummary> scans;
    for (const auto& object : extract_items_objects(json)) {
        const std::string id = string_field(object, "id");
        if (id.empty()) continue;
        ScanSummary summary{id, string_field(object, "title"), int_field(object, ...
        "chapterCount"), int_field(object, "pageCount")};
        summary.progress = {int_field(object, "progressChapter", 1), int_field(object, ...
        "progressPage", 1)};
        summary.progress.normalize();
        summary.favorite = bool_field(object, "favorite");
        summary.lastReadAt = string_field(object, "lastReadAt");
    }
}

```

```

        summary.coverImageUrl = string_field(object, "coverImageUrl");
        scans.push_back(summary);
    }
    return scans;
}

std::vector<int> ApiScanDataSource::list_chapters(const std::string& scanId) const {
    const std::string json = http_.get_text(endpoint("/api/scans/" + url_encode(scanId) + ...
        "/chapters"));
    std::vector<int> chapters;
    for (const auto& object : extract_items_objects(json)) {
        const int chapter = int_field(object, "chapter");
        if (chapter > 0) chapters.push_back(chapter);
    }
    return chapters;
}

std::vector<ScanPageInfo> ApiScanDataSource::list_pages(const std::string& scanId, int chapter) ...
const {
    const std::string json = http_.get_text(endpoint("/api/scans/" + url_encode(scanId) + ...
        "/chapters/" + std::to_string(chapter) + "/pages"));
    std::vector<ScanPageInfo> pages;
    for (const auto& object : extract_items_objects(json)) {
        ScanPageInfo page;
        page.chapter = int_field(object, "chapter");
        page.page = int_field(object, "page");
        page.mimeType = string_field(object, "mimeType");
        page.sizeBytes = long_field(object, "sizeBytes");
        page.imageUrl = string_field(object, "imageUrl");
        if (page.chapter > 0 && page.page > 0) pages.push_back(page);
    }
    return pages;
}

std::filesystem::path ApiScanDataSource::materialize_page(const std::string& scanId, ...
    ScanProgress progress) const {
    progress.normalize();
    const auto pages = list_pages(scanId, progress.chapter);
    auto it = std::find_if(pages.begin(), pages.end(), [&](const ScanPageInfo& page) { return ...
        page.page == progress.page; });
    if (it == pages.end()) throw std::runtime_error("Page introuvable via l'API: " + scanId);

    const auto output = cacheRoot_ / scanId / std::to_string(progress.chapter) / ...
        (std::to_string(progress.page) + extension_for_mime(it->mimeType));
    if (!std::filesystem::exists(output)) http_.download_file(endpoint(it->imageUrl), output);
    return output;
}

ScanProgress ApiScanDataSource::load_progress(const std::string& scanId, const std::string& ...
    profile) const {
    const std::string json = http_.get_text(endpoint("/api/scans/" + url_encode(scanId) + ...
        "/progress?profile=" + url_encode(profile)));
    ScanProgress progress{int_field(json, "chapter", 1), int_field(json, "page", 1)};
    progress.normalize();
    return progress;
}

void ApiScanDataSource::save_progress(const std::string& scanId, ScanProgress progress, const ...
    std::string& profile) const {
    progress.normalize();
    const std::string body = "{\"chapter\": " + std::to_string(progress.chapter) + ", \"page\": " ...
        + std::to_string(progress.page) + "}";
    (void)http_.post_json(endpoint("/api/scans/" + url_encode(scanId) + "/progress?profile=" + ...
        url_encode(profile)), body);
}

std::vector<ProfileSummary> ApiScanDataSource::list_profiles() const {
    const std::string json = http_.get_text(endpoint("/api/profiles"));
    std::vector<ProfileSummary> profiles;
    for (const auto& object : extract_items_objects(json)) {
        const std::string id = string_field(object, "id");
        if (!id.empty()) profiles.push_back({id, string_field(object, "displayName", id), ...
            string_field(object, "avatarColor", "#3b82f6")});
    }
    if (profiles.empty()) profiles.push_back({"default", "Default", "#3b82f6"});
    return profiles;
}

void ApiScanDataSource::save_profile(const ProfileSummary& profile) const {
    const std::string id = profile.id.empty() ? "default" : profile.id;
    const std::string body = "{\"id\": \"" + scngui::json::escape(id) + "\", \"displayName\": \"" ...
        + scngui::json::escape(profile.displayName.empty() ? id : profile.displayName) + ...
        "\"\", \"avatarColor\": \"" + scngui::json::escape(profile.avatarColor.empty() ? "#3b82f6" : ...
        profile.avatarColor) + "\"}";
    (void)http_.post_json(endpoint("/api/profiles"), body);
}

std::vector<std::string> ApiScanDataSource::list_favorites(const std::string& profile) const {
    const std::string json = http_.get_text(endpoint("/api/profiles/" + url_encode(profile) + ...
        "/favorites"));
    std::vector<std::string> favorites;
    auto root = scngui::json::parse(json);
    if (!root) return favorites;
    const auto* items = scngui::json::find_field(*root, "items");
    if (items == nullptr || !items->is_array()) return favorites;
    for (const auto& item : items->as_array()) if (item.is_string()) ...
        favorites.push_back(item.as_string());
    return favorites;
}

void ApiScanDataSource::set_favorite(const std::string& scanId, bool favorite, const ...
    std::string& profile) const {
    const std::string body = std::string("{\"favorite\": " + (favorite ? "true" : "false") + ...

```

```

    };
    (void)http_.post_json(endpoint("/api/scans/" + url_encode(scanId) + "/favorite?profile=" + ...
    url_encode(profile)), body);
}

std::vector<BookmarkSummary> ApiScanDataSource::list_bookmarks(const std::string& scanId, const ...
std::string& profile) const {
    const std::string json = http_.get_text(endpoint("/api/scans/" + url_encode(scanId) + ...
    "/bookmarks?profile=" + url_encode(profile)));
    std::vector<BookmarkSummary> bookmarks;
    for (const auto& object : extract_items_objects(json)) {
        bookmarks.push_back({long_field(object, "id"), string_field(object, "scanId", scanId), ...
        int_field(object, "chapter", 1), int_field(object, "page", 1), string_field(object, ...
        "note"), string_field(object, "createdAt"))});
    }
    return bookmarks;
}

BookmarkSummary ApiScanDataSource::add_bookmark(const std::string& scanId, ScanProgress ...
progress, const std::string& note, const std::string& profile) const {
    progress.normalize();
    const std::string body = "{\\\"chapter\\\": \" + std::to_string(progress.chapter) + \",\\\"page\\\": \" + ...
    + std::to_string(progress.page) + \",\\\"note\\\": \" + scangui::json::escape(note) + \"\\\"}\"";
    const std::string json = http_.post_json(endpoint("/api/scans/" + url_encode(scanId) + ...
    "/bookmarks?profile=" + url_encode(profile)), body);
    return {long_field(json, "id"), string_field(json, "scanId", scanId), int_field(json, ...
    "chapter", progress.chapter), int_field(json, "page", progress.page), string_field(json, ...
    "note", note), string_field(json, "createdAt")};
}

std::vector<HistoryEntry> ApiScanDataSource::list_history(const std::string& profile, int ...
limit) const {
    const std::string json = http_.get_text(endpoint("/api/profiles/" + url_encode(profile) + ...
    "/history?limit=" + std::to_string(limit)));
    std::vector<HistoryEntry> history;
    for (const auto& object : extract_items_objects(json)) {
        history.push_back({string_field(object, "scanId"), string_field(object, "title"), ...
        int_field(object, "chapter", 1), int_field(object, "page", 1), string_field(object, ...
        "readAt")});
    }
    return history;
}

std::vector<SearchResultSummary> ApiScanDataSource::search_text(const std::string& query, int ...
limit) const {
    if (query.empty()) return {};
    const std::string json = http_.get_text(endpoint("/api/search?q=" + url_encode(query) + ...
    "&limit=" + std::to_string(limit)));
    std::vector<SearchResultSummary> results;
    for (const auto& object : extract_items_objects(json)) {
        results.push_back({string_field(object, "scanId"), string_field(object, "title"), ...
        int_field(object, "chapter", 1), int_field(object, "page", 1), string_field(object, ...
        "snippet"), "api"});
    }
    return results;
}

```

5.15 15. Routage central de l'API

Le controleur expose les routes serveur, web, progression, profils, favoris, offline, OCR et admin.

server_src/ScanApiController.cpp - lignes 128-183

```

HttpResponse ScanApiController::handle(const HttpRequest& request) {
    std::lock_guard<std::mutex> lock(requestMutex_);
    try {
        const auto parts = split_path(request.path);
        if (request.method == "GET" && (request.path == "/" || (parts.size() >= 1 && parts[0] ...
        == "web"))) return serve_web(request);
        if (request.method == "GET" && parts == std::vector<std::string>{"api", "health"}) ...
        return health();
        if (request.method == "GET" && parts == std::vector<std::string>{"api", "version"}) ...
        return version();
        if (request.method == "GET" && parts == std::vector<std::string>{"api", "stats"}) ...
        return stats();
        if (request.method == "GET" && parts == std::vector<std::string>{"api", "search"}) ...
        return search(request);
        if (request.method == "GET" && parts == std::vector<std::string>{"api", "offline", ...
        "manifest"}) return offline_manifest(request);
        if (request.method == "POST" && parts == std::vector<std::string>{"api", "admin", ...
        "sync"}) return sync_library(request);
        if (request.method == "POST" && parts == std::vector<std::string>{"api", "admin", ...
        "ocr"}) return ocr_library(request);

        if (parts.size() >= 2 && parts[0] == "api" && parts[1] == "profiles") {
            if (request.method == "GET" && parts.size() == 2) return list_profiles();
            if (request.method == "POST" && parts.size() == 2) return ...
            create_profile(request.body);
            if (request.method == "GET" && parts.size() == 4 && parts[3] == "favorites") return ...
            list_favorites(parts[2]);
            if (request.method == "GET" && parts.size() == 4 && parts[3] == "history") return ...
            list_history(parts[2], request);
        }

        if (request.method == "GET" && parts == std::vector<std::string>{"api", "scans"}) ...
    }
}

```

```

return list_scans(request);

    if (parts.size() >= 3 && parts[0] == "api" && parts[1] == "scans") {
        const auto& scanId = parts[2];
        if (request.method == "GET" && parts.size() == 3) return get_scan(scanId, request);
        if (request.method == "GET" && parts.size() == 4 && parts[3] == "chapters") return ...
list_chapters(scanId);
        if (request.method == "GET" && parts.size() == 6 && parts[3] == "chapters" && ...
parts[5] == "summary") {
            auto chapter = parse_int(parts[4]);
            if (!chapter) return HttpResponse::bad_request("Invalid chapter number");
            return chapter_summary(scanId, *chapter);
        }
        if (request.method == "GET" && parts.size() == 6 && parts[3] == "chapters" && ...
parts[5] == "pages") {
            auto chapter = parse_int(parts[4]);
            if (!chapter) return HttpResponse::bad_request("Invalid chapter number");
            return list_pages(scanId, *chapter, request);
        }
        if (request.method == "GET" && parts.size() == 8 && parts[3] == "chapters" && ...
parts[5] == "pages" && parts[7] == "image") {
            auto chapter = parse_int(parts[4]);
            auto page = parse_int(parts[6]);
            if (!chapter || !page) return HttpResponse::bad_request("Invalid chapter or ...
page number");
            return get_image(scanId, *chapter, *page);
        }
        if ((request.method == "GET" || request.method == "POST" || request.method == ...
"PUT") && parts.size() == 4 && parts[3] == "progress") {
            const std::string profile = query_string(request, "profile", "default");
            return request.method == "GET" ? get_progress(scanId, profile) : ...
save_progress(scanId, profile, request.body);
        }
        if ((request.method == "POST" || request.method == "PUT") && parts.size() == 4 && ...
parts[3] == "favorite") return set_favorite(scanId, query_string(request, "profile", ...
"default"), request.body);
        if (request.method == "GET" && parts.size() == 4 && parts[3] == "bookmarks") return ...
list_bookmarks(scanId, query_string(request, "profile", "default"));
        if (request.method == "POST" && parts.size() == 4 && parts[3] == "bookmarks") ...
return add_bookmark(scanId, query_string(request, "profile", "default"), request.body);
    }

    return HttpResponse::not_found();
} catch (const std::exception& error) {
    return HttpResponse::server_error(error.what());
}
}

```

5.16 16. API de bibliotheque paginee et filtree

Les parametres q, favorites, sort, limit et offset preparent une vraie bibliotheque client.

server_src/ScanApiController.cpp - lignes 231-278

```

HttpResponse ScanApiController::list_scans(const HttpRequest& request) const {
    const std::string profile = query_string(request, "profile", "default");
    const std::string q = lower_ascii(query_string(request, "q", ""));
    const bool favoritesOnly = query_string(request, "favorites", "0") == "1";
    const std::string sort = query_string(request, "sort", "title");
    int limit = query_int(request, "limit", 0);
    int offset = query_int(request, "offset", 0);

    auto scans = database_.list_scans(profile);
    scans.erase(std::remove_if(scans.begin(), scans.end(), [&](const ScanSummaryRecord& scan) {
        if (favoritesOnly && !scan.favorite) return true;
        if (!q.empty() && lower_ascii(scan.title).find(q) == std::string::npos && ...
lower_ascii(scan.id).find(q) == std::string::npos) return true;
        return false;
    }), scans.end());

    if (sort == "progress") {
        std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.progressChapter, a.progressPage, a.title) > std::tie(b.progressChapter, ...
b.progressPage, b.title); });
    } else if (sort == "chapters") {
        std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.chapterCount, a.title) > std::tie(b.chapterCount, b.title); });
    } else if (sort == "pages") {
        std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.pageCount, a.title) > std::tie(b.pageCount, b.title); });
    } else if (sort == "lastRead") {
        std::sort(scans.begin(), scans.end(), [](const auto& a, const auto& b) { return ...
std::tie(a.lastReadAt, a.title) > std::tie(b.lastReadAt, b.title); });
    }

    const int total = static_cast<int>(scans.size());
    if (offset < 0) offset = 0;
    if (offset > total) offset = total;
    auto begin = scans.begin() + offset;
    auto end = scans.end();
    if (limit > 0 && offset + limit < total) end = scans.begin() + offset + limit;

    std::string json = "{\"profile\":\"" + scangui::json::escape(profile) + "\",\"total\": " + ...
std::to_string(total) + ", \"items\":[ ";
    bool first = true;

```

```

    for (auto it = begin; it != end; ++it) {
        if (!first) json += ',';
        first = false;
        json += scan_to_json(*it);
    }
    json += "]}";
    return HttpResponse::json(200, json);
}

HttpResponse ScanApiController::get_scan(const std::string& scanId, const HttpRequest& request) ...
const {
    auto scan = database_.get_scan(scanId, query_string(request, "profile", "default"));
    return scan ? HttpResponse::json(200, scan_to_json(*scan)) : HttpResponse::not_found("Scan ...
    not found");
}

```

5.17 17. Recherche OCR et resume de chapitre

Ces routes interrogent les textes indexes pour retrouver une page ou produire un extrait de chapitre.

server_src/ScanApiController.cpp - lignes 417-443

```

HttpResponse ScanApiController::search(const HttpRequest& request) const {
    const std::string query = query_string(request, "q", "");
    if (query.empty()) return HttpResponse::bad_request("Missing q query parameter");
    const auto rows = database_.search_page_text(query, query_int(request, "limit", 50));
    std::string json = "{ \"query\": \"" + scangui::json::escape(query) + "\", \"items\": [";
    for (std::size_t i = 0; i < rows.size(); ++i) {
        if (i) json += ',';
        const auto& row = rows[i];
        json += "{ \"scanId\": \"" + scangui::json::escape(row.scanId) + "
            + "\", \"title\": \"" + scangui::json::escape(row.title) + "
            + "\", \"chapter\": \"" + std::to_string(row.chapter) + "
            + "\", \"page\": \"" + std::to_string(row.page) + "
            + "\", \"snippet\": \"" + scangui::json::escape(row.snippet) + "\" }";
    }
    json += "]}";
    return HttpResponse::json(200, json);
}

HttpResponse ScanApiController::chapter_summary(const std::string& scanId, int chapter) const {
    if (!database_.get_scan(scanId)) return HttpResponse::not_found("Scan not found");
    const auto texts = database_.chapter_text(scanId, chapter);
    std::string joined;
    for (const auto& pageText : texts) {
        if (!joined.empty()) joined += "\n";
        joined += pageText.text;
    }
    return HttpResponse::json(200, "{ \"scanId\": \"" + scangui::json::escape(scanId) + "
        + "\", \"chapter\": \"" + std::to_string(chapter) + "\", \"summary\": \"" + ...
        + scangui::json::escape(summarize_text(joined)) + "\" }");
}

```

5.18 18. Securite admin locale et token optionnel

Les operations dangereuses sont refusees hors localhost et peuvent exiger un token.

server_src/ScanApiController.cpp - lignes 464-479

```

bool ScanApiController::is_safe_page_path(const std::filesystem::path& path) const {
    if (!std::filesystem::exists(path)) return false;
    const auto root = std::filesystem::weakly_canonical(scanRoot_);
    const auto file = std::filesystem::weakly_canonical(path);
    const std::string rootText = root.string();
    const std::string fileText = file.string();
    return fileText == rootText || fileText.rfind(rootText + std::string(1, ...
    std::filesystem::path::preferred_separator), 0) == 0;
}

bool ScanApiController::admin_request_allowed(const HttpRequest& request) const {
    const bool local = bindHost_ == "127.0.0.1" || bindHost_ == "localhost";
    if (!local) return false;
    if (adminToken_.empty()) return true;
    auto it = request.headers.find("x-scangui-admin-token");
    return it != request.headers.end() && it->second == adminToken_;
}

```

5.19 19. Schema PostgreSQL enrichi

Le schema gere profils, progression, favoris, marque-pages, historique et textes OCR.

migrations/001_init.sql - lignes 25-87

```

PRIMARY KEY (scan_id, chapter_number, page_number),
FOREIGN KEY (scan_id, chapter_number) REFERENCES chapters(scan_id, chapter_number) ON ...
DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS profiles (
  id TEXT PRIMARY KEY,
  display_name TEXT NOT NULL,
  avatar_color TEXT NOT NULL DEFAULT '#3b82f6',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO profiles(id, display_name, avatar_color)
VALUES('default', 'Default', '#3b82f6')
ON CONFLICT(id) DO NOTHING;

CREATE TABLE IF NOT EXISTS reading_progress (
  scan_id TEXT NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  profile TEXT NOT NULL DEFAULT 'default',
  chapter_number INTEGER NOT NULL CHECK (chapter_number > 0),
  page_number INTEGER NOT NULL CHECK (page_number > 0),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (scan_id, profile)
);

CREATE TABLE IF NOT EXISTS favorites (
  scan_id TEXT NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  profile TEXT NOT NULL DEFAULT 'default',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (scan_id, profile)
);

CREATE TABLE IF NOT EXISTS bookmarks (
  id BIGSERIAL PRIMARY KEY,
  scan_id TEXT NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  profile TEXT NOT NULL DEFAULT 'default',
  chapter_number INTEGER NOT NULL CHECK (chapter_number > 0),
  page_number INTEGER NOT NULL CHECK (page_number > 0),
  note TEXT NOT NULL DEFAULT '',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS reading_history (
  id BIGSERIAL PRIMARY KEY,
  scan_id TEXT NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  profile TEXT NOT NULL DEFAULT 'default',
  chapter_number INTEGER NOT NULL CHECK (chapter_number > 0),
  page_number INTEGER NOT NULL CHECK (page_number > 0),
  read_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS page_texts (
  scan_id TEXT NOT NULL,
  chapter_number INTEGER NOT NULL,
  page_number INTEGER NOT NULL,
  page_text TEXT NOT NULL DEFAULT '',
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (scan_id, chapter_number, page_number),
  FOREIGN KEY (scan_id, chapter_number, page_number) REFERENCES pages(scan_id, ...
  chapter_number, page_number) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_pages_scan_chapter ON pages(scan_id, chapter_number, ...
  page_number);
CREATE INDEX IF NOT EXISTS idx_history_profile ON reading_history(profile, read_at DESC);
CREATE INDEX IF NOT EXISTS idx_page_texts_scan_chapter ON page_texts(scan_id, chapter_number, ...
  page_number);

```

5.20 20. Indexeur OCR local / sidecar / Tesseract

L'OCR privilégie les fichiers .txt voisins des images et peut appeler un moteur externe configuré.

server_src/OcrIndexer.cpp - lignes 53-115

```

OcrIndexReport OcrIndexer::index_all() {
  OcrIndexReport total;
  for (const auto& scan : database_.list_scans("default")) {
    auto report = index_scan(scan.id);
    total.pagesVisited += report.pagesVisited;
    total.indexedPages += report.indexedPages;
    total.skippedPages += report.skippedPages;
    total.failedPages += report.failedPages;
  }
  return total;
}

OcrIndexReport OcrIndexer::index_scan(const std::string& scanId) {
  OcrIndexReport report;
  for (const auto& chapter : database_.list_chapters(scanId)) {
    for (const auto& page : database_.list_pages(scanId, chapter.number)) {

```

```

        ++report.pagesVisited;
        try {
            const std::string text = extract_text(page);
            if (!is_text_useful(text)) {
                ++report.skippedPages;
                continue;
            }
            database_.upsert_page_text(scanId, page.chapter, page.page, text);
            ++report.indexedPages;
        } catch (...) {
            ++report.failedPages;
        }
    }
}

return report;
}

std::string OcrIndexer::extract_text(const PageRecord& page) const {
    std::string text = read_sidecar_text(page.filePath);
    if (is_text_useful(text)) return text;
    if (!commandTemplate_.empty()) return run_ocr_command(page.filePath);
    return {};
}

std::string OcrIndexer::read_sidecar_text(const std::filesystem::path& imagePath) const {
    std::filesystem::path txt = imagePath;
    txt.replace_extension(".txt");
    return read_file(txt);
}

std::string OcrIndexer::run_ocr_command(const std::filesystem::path& imagePath) const {
    const auto output = std::filesystem::temp_directory_path() / ("scangui-ocr-" + ...
    std::to_string(std::rand()) + ".txt");
    std::string command = commandTemplate_;
    if (command.find("{input}") != std::string::npos || command.find("{output}") != ...
    std::string::npos) {
        command = replace_all(command, "{input}", shell_quote(imagePath));
        command = replace_all(command, "{output}", shell_quote(output));
    } else {
        command += " " + shell_quote(imagePath) + " " + shell_quote(output);
    }
    const int exitCode = std::system(command.c_str());
    if (exitCode != 0) return {};
    std::string text = read_file(output);
    std::filesystem::remove(output);
    return text;
}

```

5.21 21. Client web local minimal

Le client web consomme la meme API que GTK et prouve l'architecture multi-clients.

web/app.js - lignes 1-124

```

const state = { scans: [], current: null, chapters: [], pages: [], chapterIndex: 0, pageIndex: ...
0 };
const $ = (id) => document.getElementById(id);
const profile = () => $('profile').value || 'default';
const api = async (url, options = {}) => {
    const response = await fetch(url, options);
    if (!response.ok) throw new Error(await response.text());
    return response.json();
};

async function loadLibrary() {
    const params = new URLSearchParams({ profile: profile(), sort: $('sort').value });
    if ($('search').value) params.set('q', $('search').value);
    if ($('favoritesOnly').checked) params.set('favorites', '1');
    const data = await api('/api/scans?' + params);
    state.scans = data.items || [];
    renderLibrary();
    if ($('search').value) searchOcr($('search').value).catch(() => {});
}

function renderLibrary() {
    $('library').innerHTML = '';
    state.scans.forEach(scan => {
        const card = document.createElement('div');
        card.className = 'card';
        card.innerHTML = `<div><h3>${escapeHtml(scan.title)}</h3><div class="badge">${scan.favorite ...
? 'Favori' : 'Lecture'}</div></div>
<p>${scan.chapterCount} chap. - ${scan.pageCount} pages<br>Progression: ch. ...
${scan.progressChapter || 1}, p. ${scan.progressPage || 1}<br>${scan.lastReadAt || ''}</p>`;
        card.onclick = () => openScan(scan, scan.progressChapter || 1, scan.progressPage || 1);
        $('library').appendChild(card);
    });
}

async function openScan(scan, chapter = 1, page = 1) {
    state.current = scan;
    $('readerTitle').textContent = scan.title;
    const chaptersData = await api('/api/scans/' + encodeURIComponent(scan.id) + '/chapters');
    state.chapters = (chaptersData.items || []).map(c => c.chapter);
    state.chapterIndex = Math.max(0, state.chapters.indexOf(chapter));
    await loadPages(page);
}

```

```

}

async function loadPages(page = 1) {
  if (!state.current || state.chapters.length === 0) return;
  const chapter = state.chapters[state.chapterIndex];
  const pagesData = await api(`/api/scans/${encodeURIComponent(state.current.id)}/chapters/${chapterIndex}/pages`);
  state.pages = pagesData.items || [];
  state.pageIndex = Math.max(0, state.pages.findIndex(p => p.page === page));
  if (state.pageIndex < 0) state.pageIndex = 0;
  renderPage();
}

async function renderPage() {
  const page = state.pages[state.pageIndex];
  if (!state.current || !page) return;
  $('page').src = page.imageUrl;
  $('page').classList.remove('hidden');
  $('meta').textContent = `Chapitre ${page.chapter} - page ${page.page} / ...`;
  await api(`/api/scans/${encodeURIComponent(state.current.id)}/progress?profile=${encodeURIComponent(profile())}`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ ...chapter, page: page.chapter, page: page.page })
  }).catch(() => {});
}

async function nextPage() {
  if (state.pageIndex + 1 < state.pages.length) { state.pageIndex++; return renderPage(); }
  if (state.chapterIndex + 1 < state.chapters.length) { state.chapterIndex++; return loadPages(1); }
}

async function prevPage() {
  if (state.pageIndex > 0) { state.pageIndex--; return renderPage(); }
  if (state.chapterIndex > 0) { state.chapterIndex--; await loadPages(999999); state.pageIndex = Math.max(0, state.pages.length - 1); return renderPage(); }
}

async function toggleFavorite() {
  if (!state.current) return;
  state.current.favorite = !state.current.favorite;
  await api(`/api/scans/${encodeURIComponent(state.current.id)}/favorite?profile=${encodeURIComponent(profile())}`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ ...state.current, favorite: state.current.favorite })
  });
  renderLibrary();
}

async function addBookmark() {
  if (!state.current || !state.pages[state.pageIndex]) return;
  const p = state.pages[state.pageIndex];
  const note = prompt('Note du marque-page', '') || '';
  await api(`/api/scans/${encodeURIComponent(state.current.id)}/bookmarks?profile=${encodeURIComponent(profile())}`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ ...p, chapter: p.chapter, page: p.page, note })
  });
}

async function resume() {
  const data = await api(`/api/profiles/${encodeURIComponent(profile())}/history?limit=1`);
  const item = (data.items || [])[0];
  if (!item) return;
  const scan = state.scans.find(s => s.id === item.scanId) || { id: item.scanId, title: item.title };
  openScan(scan, item.chapter, item.page);
}

async function searchOcr(query) {
  const data = await api(`/api/search?q=${encodeURIComponent(query)}&limit=8`);
  $('ocrResults').innerHTML = '';
  (data.items || []).forEach(item => {
    const div = document.createElement('div');
    div.className = 'result';
    div.innerHTML = `<strong>${escapeHtml(item.title)}</strong> - ch. ${item.chapter}, p. ...`;
    div.innerHTML += `${item.page}<br>${escapeHtml(item.snippet)}`;
    div.onclick = () => openScan({ id: item.scanId, title: item.title }, item.chapter, item.page);
    $('ocrResults').appendChild(div);
  });
}

function escapeHtml(text) {
  return String(text || '').replace(/&lt;|&gt;|&quot;|&#039;/g, c => ...
    ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', '&#039;': '&#039;' }[c]));
}

$('refresh').onclick = loadLibrary;
$('resume').onclick = resume;
$('next').onclick = nextPage;
$('prev').onclick = prevPage;
$('favorite').onclick = toggleFavorite;
$('bookmark').onclick = addBookmark;
$('sort').onchange = loadLibrary;
$('favoritesOnly').onchange = loadLibrary;
$('search').addEventListener('keydown', e => { if (e.key === 'Enter') loadLibrary(); });
window.addEventListener('keydown', e => { if (e.key === 'ArrowRight') nextPage(); if (e.key === 'ArrowLeft') prevPage(); });
loadLibrary().catch(err => $('library').innerHTML = `<div class="result">Erreur: ...
  ${escapeHtml(err.message)}</div>`);

```


5.22 22. Tests unitaires ajoutées

La suite autonome couvre les briques les plus critiques ajoutées dans cette version.

tests/run_unit_tests.cpp - lignes 1-103

```
#include "application/DownloadQueue.hpp"
#include "application/FileSystemScanDataSource.hpp"
#include "application/OfflineLibrarySync.hpp"
#include "domain/ScanSession.hpp"
#include "infrastructure/JsonScanRepository.hpp"
#include "infrastructure/SimpleJson.hpp"
#include "server/HttpTypes.hpp"

#include <atomic>
#include <cassert>
#include <chrono>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <thread>
#include <unistd.h>

namespace fs = std::filesystem;

static void touch(const fs::path& path) {
    fs::create_directories(path.parent_path());
    std::ofstream out(path, std::ios::binary);
    out << "x";
}

static fs::path temp_root(const std::string& suffix = "") {
    auto root = fs::temp_directory_path() / ("scangui-tests-" + std::to_string(::getpid()) + ...
    suffix);
    fs::remove_all(root);
    fs::create_directories(root);
    return root;
}

int main() {
    const std::string json = R"({"title": "Dr ...
    \"Stone\", \"chapter\": 12, \"favorite\": true, \"items\": [{\"id\": \"a\"}, {\"id\": \"b\"}]}");
    assert(scangui::json::string_field(json, "title") == "Dr \"Stone\"");
    assert(scangui::json::int_field(json, "chapter") == 12);
    assert(scangui::json::extract_bool(json, "favorite").value_or(false));
    assert(scangui::json::extract_object_array(json).size() == 2);

    auto parts = split_path("/api/scans/one%20piece/chapters");
    assert(parts.size() == 4 && parts[2] == "one piece");
    assert(guess_mime_type("1.PNG") == "image/png");

    auto root = temp_root();
    touch(root / "scan-a" / "1" / "1.jpg");
    touch(root / "scan-a" / "1" / "2.png");
    touch(root / "scan-a" / "2" / "1.webp");

    ScanSession session(root / "scan-a");
    assert(session.current_page().has_value());
    assert(session.next_page()->progress.page == 2);
    assert(session.next_page()->progress.chapter == 2);

    JsonScanRepository repo;
    repo.save_progress(root / "scan-a", ScanProgress{2, 1});
    auto loaded = repo.load(root / "scan-a");
    assert(loaded.has_value() && loaded->saveProgress.chapter == 2);

    FileSystemScanDataSource source(root);
    auto scans = source.list_scans();
    assert(scans.size() == 1 && scans[0].pageCount == 3);
    source.set_favorite("scan-a", true, "tester");
    assert(source.list_scans("tester")[0].favorite);
    source.add_bookmark("scan-a", ScanProgress{1, 2}, "note", "tester");
    assert(source.list_bookmarks("scan-a", "tester").size() == 1);
    touch(root / "scan-a" / "ocr" / "1" / "2.txt");
    { std::ofstream txt(root / "scan-a" / "ocr" / "1" / "2.txt"); txt << "recherche speciale ...
    dans une bulle"; }
    assert(!source.search_text("speciale", 5).empty());

    auto offlineRoot = temp_root("-offline");
    OfflineLibrarySync offline(source, offlineRoot);
    auto report = offline.sync_all("tester");
    assert(report.scans == 1);
    assert(fs::exists(offlineRoot / "scan-a" / "1" / "1.jpg"));

    std::atomic<int> completed{0};
    std::atomic<int> progressEvents{0};
    DownloadQueue queue([&](const DownloadJobReport& jobReport) {
        assert(jobReport.success);
        ++completed;
    }, [&](const std::string&, int, int, const std::string&) { ++progressEvents; });
    queue.start();
    queue.pause();
    queue.push(DownloadJob{"paused", [&](DownloadJobContext& ctx) { assert(!ctx.cancelled()); ...
    ctx.report(1, 1, "paused job"); touch(root / "queue" / "paused.txt"); }, 0});
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
    assert(queue.paused());
    assert(queue.pending_jobs() == 1);
    assert(!fs::exists(root / "queue" / "paused.txt"));
    queue.resume();
    queue.push(DownloadJob{"unit", [&](DownloadJobContext& ctx) { assert(!ctx.cancelled()); ...
```

```

ctx.report(1, 1, "unit job"); touch(root / "queue" / "done.txt"); }, 0));
for (int i = 0; i < 100 && completed.load() < 2; ++i) ...
std::this_thread::sleep_for(std::chrono::milliseconds(10));
queue.stop();
assert(completed.load() == 2);
assert(progressEvents.load() >= 2);
assert(fs::exists(root / "queue" / "paused.txt"));
assert(fs::exists(root / "queue" / "done.txt"));

fs::remove_all(root);
fs::remove_all(offlineRoot);
fs::remove_all("cache/profiles/tester");
std::cout << "All ScanGUI unit tests passed" << std::endl;
return 0;
}

```

6 Verification realisee

Verification	Resultat
make tests	OK - All ScanGUI unit tests passed
make server	OK - serveur C++ lie avec libpq
CMake sans GUI avec clang++/lld	OK - ctest passe
Compilation GTK	Limite - non executable ici : gtkmm-3.0 absent du sandbox
PDF	OK - compile et rendu en images pour controle visuel

Compilation GUI

Pour compiler la GUI sur Debian/Ubuntu, installer par exemple `libgtkmm-3.0-dev`, puis lancer `make`. L'environnement de validation ne contient pas cette dependance graphique, donc seuls les tests non-GUI, le serveur et CMake sans GUI ont ete executes ici.

7 Limites restantes assumees

La version v6 rend toutes les evolutions utilisables ou branchees dans le code, mais certaines restent volontairement simples : l'OCR ne fait pas de segmentation IA avantee des bulles, le JSON reste un parseur interne simple plutot qu'une dependance nlohmann/json, et le client web est un client local léger plutot qu'une application React complete. Ces choix gardent le projet compilable et pedagogique tout en montrant clairement ou placer une evolution industrielle.